

Rapport des Tests

Monitoring Réseau Client–Serveur avec Sockets TCP & UDP

Étudiant(s) : Hamdi Mohamed Aziz

Maddouri Ferdaous

Filière : RT2

Enseignant : Dr. Kamel Karoui

Table des matières

1	Introduction	2
2	Tests Obligatoires	2
2.1	T1 — Connexion d'un seul client	2
2.2	T2 — Connexion de plusieurs clients simultanément	3
2.3	T3 — Envoi d'un message mal formé	3
2.4	T4 — Arrêt brutal d'un client	4
2.5	T5 — Calcul correct des moyennes	4
2.6	T6 — Arrêt propre d'un client	5
2.7	T7 — Arrêt du serveur	5
2.8	T8 — Tentative de connexion à un serveur inactif	6
3	Tests des Extensions	6
3.1	T9 — Communication UDP	6
3.2	T10 — Détection d'un agent inactif	7
3.3	T11 — Simulation d'attaque flood	7
3.4	T12 — Sécurité — Ownership check	8
4	Conclusion	8

1. Introduction

Ce document présente le rapport des tests réalisés sur l'application de monitoring réseau client–serveur. Chaque test est documenté avec ses conditions d'exécution, les résultats attendus, les résultats obtenus et un verdict final.

Les tests ont été exécutés de deux façons :

- **Mode automatique** — via la suite `run_tests.bat` (aucune installation requise, aucun venv nécessaire)
- **Mode manuel** — en activant un environnement virtuel (`venv`) et en lançant les composants individuellement dans des terminaux séparés.

2. Tests Obligatoires

2.1. T1 — Connexion d'un seul client

Champ	Détail
Objectif	Vérifier qu'un seul agent peut se connecter, s'enregistrer et envoyer des rapports périodiques.
Préconditions	Serveur TCP démarré sur le port 5000.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [1] 3. Observer les fenêtres ouvertes automatiquement Manuel : 1. Lancer <code>python serveurTCP.py</code> 2. Lancer <code>python clientTCP.py</code> dans un second terminal 3. Observer les messages HELLO et REPORT
Résultat attendu	Le serveur affiche la connexion, reçoit les REPORT et répond OK à chaque message.
Résultat obtenu	Le serveur affiche <code>[+] Connection from ('127.0.0.1', port)</code> suivi de <code>[HELLO]</code> Agent enregistré, puis les rapports toutes les 5 secondes avec réponse OK. Le dashboard Tkinter affiche l'agent actif avec ses métriques.
Verdict	PASS

2.2. T2 — Connexion de plusieurs clients simultanément

Champ	Détail
Objectif	Vérifier que le serveur gère plusieurs agents simultanément sans blocage.
Préconditions	Serveur TCP démarré sur le port 5000.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [2] 3. Observer les fenêtres ouvertes automatiquement Manuel : 1. Lancer <code>python serveurTCP.py</code> 2. Ouvrir 3 terminaux et lancer <code>python clientTCP.py</code> dans chacun 3. Observer les statistiques globales dans le dashboard
Résultat attendu	Le serveur affiche les connexions de chaque agent et calcule les moyennes sur tous les agents actifs.
Résultat obtenu	3 agents connectés simultanément avec des UUID distincts. Statistiques affichées correctement toutes les 5 secondes dans le dashboard Tkinter.
Verdict	PASS

2.3. T3 — Envoi d'un message mal formé

Champ	Détail
Objectif	Vérifier que le serveur valide correctement les messages et retourne ERROR pour tout message non conforme.
Préconditions	Serveur TCP démarré.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [3] 3. Observer les fenêtres ouvertes automatiquement Manuel : 1. Lancer <code>python customClient.py</code> 2. Saisir un message invalide : <pre>REPORT agent1 0 999 1024 (cpu > 100) BONJOUR agent1 (commande inconnue) REPORT (message incomplet)</pre> 3. Observer la réponse du serveur
Résultat attendu	Le serveur retourne ERROR pour chaque message invalide sans planter.
Résultat obtenu	ERROR InvalidMessageFormat retourné pour les commandes inconnues ; ERROR InvalidData pour les valeurs CPU/RAM hors limites. Après <code>MAX_TRIES = 5</code> rejets consécutifs, <code>customClient.py</code> abandonne automatiquement.
Verdict	PASS

2.4. T4 — Arrêt brutal d'un client

Champ	Détail
Objectif	Vérifier que le serveur continue de fonctionner normalement après la déconnexion brutale d'un agent.
Préconditions	Au moins un agent connecté au serveur.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [4] 3. Observer les fenêtres ouvertes automatiquement Manuel : 1. Lancer le serveur et deux clients 2. Fermer brutalement un terminal client (sans <code>Ctrl+C</code>) 3. Attendre $3 \times T = 15$ secondes 4. Observer le comportement du serveur
Résultat attendu	Le serveur détecte la déconnexion, affiche une alerte d'inactivité après 15 secondes et continue de servir les autres agents.
Résultat obtenu	Le serveur affiche [-] <code>Connection closed</code> immédiatement via le bloc <code>finally</code> . Après 15 secondes : [ALERT] <code>Agent X is INACTIVE!</code> puis [INFO] <code>Agent removed from stats</code> . Les autres agents continuent normalement.
Verdict	PASS

2.5. T5 — Calcul correct des moyennes

Champ	Détail
Objectif	Vérifier que le serveur calcule correctement les moyennes CPU et RAM sur l'ensemble des agents actifs.
Préconditions	Au moins deux agents connectés et envoyant des <code>REPORT</code> .
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner n'importe quel test [1-10] 3. Vérifier les moyennes dans le dashboard Tkinter Manuel : 1. Connecter deux agents 2. Observer les statistiques après chaque cycle de T secondes dans le dashboard 3. Vérifier manuellement que les moyennes sont correctes
Résultat attendu	Les moyennes affichées correspondent à la moyenne arithmétique des valeurs reçues de tous les agents <code>verified = True</code> .
Résultat obtenu	Les valeurs CPU et RAM affichées dans le dashboard ont été vérifiées et correspondent aux calculs manuels. Seuls les agents ayant reçu au moins un <code>REPORT</code> valide (<code>verified = True</code>) sont inclus dans le calcul.
Verdict	PASS

2.6. T6 — Arrêt propre d'un client

Champ	Détail
Objectif	Vérifier que le client envoie un message BYE avant de se déconnecter via Ctrl+C .
Préconditions	Un agent connecté au serveur.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner n'importe quel test [1-10] 3. Fermer la fenêtre client et observer le terminal serveur Manuel : 1. Lancer un client 2. Appuyer sur Ctrl+C 3. Observer le terminal du client et du serveur
Résultat attendu	Le client envoie BYE et le serveur retire immédiatement l'agent sans attendre le délai d'inactivité.
Résultat obtenu	Le client affiche [TCP CLIENT] Disconnected et le serveur affiche [BYE] Agent disconnected properly suivi de [-] Connection closed. L'agent disparaît immédiatement du dashboard.
Verdict	PASS

2.7. T7 — Arrêt du serveur

Champ	Détail
Objectif	Vérifier que les clients détectent l'arrêt du serveur et s'arrêtent proprement.
Préconditions	Serveur et au moins un client en cours d'exécution.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner n'importe quel test [1-10] 3. Fermer la fenêtre serveur et observer le client Manuel : 1. Lancer le serveur et un client 2. Fermer la fenêtre Tkinter du serveur (ou Ctrl+C) 3. Observer le comportement du client
Résultat attendu	Le client détecte la perte de connexion et affiche un message d'erreur avant de s'arrêter.
Résultat obtenu	Le client affiche : [ERROR] Connection lost suivi de [TCP CLIENT] Server unreachable. Exiting. Le serveur affiche : [TCP SERVER] Shutting down.
Verdict	PASS

2.8. T8 — Tentative de connexion à un serveur inactif

Champ	Détail
Objectif	Vérifier que le client gère correctement l'impossibilité de se connecter à un serveur non démarré.
Préconditions	Serveur non démarré.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [5] 3. Observer la fenêtre ouverte automatiquement Manuel : 1. Lancer <code>python clientTCP.py</code> sans serveur 2. Observer le message d'erreur
Résultat attendu	Le client affiche un message d'erreur explicite et s'arrête proprement.
Résultat obtenu	Le client affiche <code>[ERROR] Could not connect to 127.0.0.1:5000. Is the server running?</code> et s'arrête immédiatement sans boucle de retry.
Verdict	PASS

3. Tests des Extensions

3.1. T9 — Communication UDP

Champ	Détail
Objectif	Vérifier que la communication UDP fonctionne correctement et démontrer la différence avec TCP.
Préconditions	Serveur UDP démarré sur le port 5001.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [7] 3. Observer les fenêtres ouvertes automatiquement Manuel : 1. <code>python serveurUDP.py</code> 2. <code>python clientUDP.py</code> 3. Observer les REPORT et les réponses 4. Arrêter le serveur et observer le comportement du client UDP
Résultat attendu	Les REPORT sont envoyés et reçus normalement. En cas de serveur inactif, le client UDP réessaie le HELLO jusqu'à <code>MAX_RETRIES = 5</code> fois puis abandonne.
Résultat obtenu	Communication fonctionnelle. En cas de serveur inactif, le client affiche <code>[WARNING] No response to HELLO, retrying (N/5)...</code> puis <code>[ERROR] Server unreachable after several attempts. Exiting.</code>
Verdict	PASS

3.2. T10 — Détection d'un agent inactif

Champ	Détail
Objectif	Vérifier que le serveur détecte et signale un agent qui n'envoie plus de <code>REPORT</code> depuis $3 \times T$ secondes.
Préconditions	Au moins un agent connecté.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner n'importe quel test [1-10] 3. Observer le terminal serveur après 15 secondes Manuel : 1. Connecter un agent 2. Arrêter l'agent sans envoyer <code>BYE</code> 3. Attendre $3 \times 5 = 15$ secondes 4. Observer le terminal du serveur
Résultat attendu	Le serveur affiche <code>[ALERT] Agent X is INACTIVE!</code> et exclut l'agent des statistiques.
Résultat obtenu	L'alerte s'affiche après 15 secondes : <code>[ALERT] Agent X is INACTIVE! (silent for 16s)</code> , suivi de <code>[INFO] Agent X removed from stats</code> . Les moyennes ne prennent plus en compte l'agent inactif.
Verdict	PASS

3.3. T11 — Simulation d'attaque flood

Champ	Détail
Objectif	Vérifier la robustesse du serveur face à un envoi massif de connexions et de messages simultanés.
Préconditions	Serveur démarré avec <code>MAX_AGENTS = 30</code> et <code>RATE_LIMIT_S = 2</code> .
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [6] 3. Observer le comportement du serveur pendant l'attaque Manuel : 1. lancer <code>python flood.py</code> 2. Observer le comportement du serveur pendant l'attaque 3. Vérifier que le serveur reste opérationnel après le flood
Résultat attendu	Le serveur enregistre au maximum 30 agents, bloque les <code>REPORT</code> envoyés trop rapidement et reste stable.
Résultat obtenu	Le serveur a enregistré exactement 30 agents, les 20 suivants ont reçu <code>[BLOCK] Agent limit (30) reached</code> . Chaque agent a eu 998 requêtes bloquées sur 1000. Le serveur est resté stable tout au long de l'attaque.
Verdict	PASS

3.4. T12 — Sécurité — Ownership check

Champ	Détail
Objectif	Vérifier qu'une connexion ne peut pas envoyer de <code>REPORT</code> ou <code>BYE</code> pour un agent qui ne lui appartient pas.
Préconditions	Serveur TCP démarré, au moins un agent enregistré.
Étapes	Automatique : 1. Lancer <code>run_tests.bat</code> 2. Sélectionner [3] 3. Observer les réponses <code>ERROR Unauthorized</code> dans la fenêtre Manuel : 1. Connecter un premier client (agent A) 2. Ouvrir une seconde connexion et envoyer <code>HELLO agentB host</code> 3. Depuis la seconde connexion, envoyer <code>REPORT agentA ...</code> 4. Observer la réponse du serveur
Résultat attendu	Le serveur refuse l'opération et retourne <code>ERROR Unauthorized</code> .
Résultat obtenu	Le serveur affiche <code>[SECURITY] addr tried to REPORT for agentA but owns agentB. Rejected.</code> et retourne <code>ERROR Unauthorized</code> . Même comportement pour <code>BYE</code> . En UDP, la table <code>addr_to_agent</code> assure la même protection.
Verdict	PASS

4. Conclusion

L'ensemble des tests obligatoires (T1–T8) et des tests d'extension (T9–T12) ont été réalisés avec succès. Le système s'est montré robuste face aux déconnexions brutales, aux messages invalides, aux tentatives d'usurpation d'identité et aux attaques par envoi massif.

Points clés validés :

- Le mécanisme de détection des agents inactifs fonctionne correctement avec un seuil de $3 \times T = 15$ secondes et l'ensemble `inactive_alerted` évite les alertes en double.
- La comparaison TCP/UDP met en évidence une différence fondamentale : le client TCP échoue immédiatement (`ConnectionRefusedError`), tandis que le client UDP réessaie jusqu'à 5 fois avant d'abandonner.
- Les mécanismes anti-flood (`MAX_AGENTS = 30` et `RATE_LIMIT_S = 2`) ont permis de maintenir la stabilité du serveur sous charge élevée (50 threads, 1 000 `REPORT` par thread).
- L'ownership check (TCP : `current_agent_id`; UDP : `addr_to_agent`) empêche toute usurpation d'identité entre agents.
- La suite `run_tests.bat` permet de reproduire tous les scénarios sans aucune configuration préalable.